

TP6 - Pipelines Dynamiques

Objectifs

A la fin de ce TP, vous serez capable de :

- Comprendre et utiliser les **regles conditionnelles** (rules:) pour controler l'execution des jobs
- Generer dynamiquement un fichier YAML et le declencher comme **pipeline enfant** (trigger)
- Controler le declenchement global du pipeline avec workflow:rules
- Optimiser l'execution avec le **DAG** (Directed Acyclic Graph) via needs:
- Rendre vos pipelines resilients avec interruptible, retry, timeout et resource_group

Pre-requis

- Avoir complete les TP1 a TP5
- Compte GitLab avec acces a un runner
- Node.js installe (v18+)
- Connaissance basique de YAML et des pipelines GitLab CI

Etape 1 : Qu'est-ce qu'un pipeline dynamique ?

La theorie

Jusqu'ici, vos pipelines etaient **statiques** : les memes jobs s'executent a chaque fois, quel que soit le contexte. C'est comme un restaurant qui sert exactement le meme menu tous les jours, meme si certains ingredients ne sont pas disponibles.

Un **pipeline dynamique**, c'est comme un menu qui change selon le jour :

- Le lundi, on propose du poisson (parce que c'est jour de marche)
- Le week-end, on ajoute le brunch
- Si le chef est absent, on retire les plats complexes

De la meme maniere, un pipeline dynamique peut :

- **Sauter les tests d'integration** si seule la documentation a change
- **Ajouter un job de deployment** uniquement sur la branche main
- **Generer des jobs a la volee** en fonction des variables d'environnement

Les deux approches

Approche	Comment	Quand l'utiliser
Regles conditionnelles (rules:)	On definit des conditions sur chaque job	Cas simples : "deployer seulement sur main"
Pipeline enfant dynamique (trigger:)	Un job genere un fichier YAML, un autre le declenche	Cas complexes : le contenu du pipeline depend du contexte

Etape 2 : Comprendre `rules:` (if, changes, exists)

`rules:if` - *Condition sur une variable*

```
deploy:
  script:
    - 'echo "Deploiement en production"'
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
```

Pourquoi ? On ne veut deployer en production que depuis la branche principale. Sur les autres branches, ce job est tout simplement ignore.

`rules:changes` - *Condition sur les fichiers modifies*

```
test-frontend:
  script:
    - npm run test:frontend
  rules:
    - changes:
      - "src/frontend/**/*"
      - "package.json"
```

Pourquoi ? Si seul le backend a change, inutile de relancer les tests frontend. Cela fait gagner du temps et des ressources.

`rules:exists` - *Condition sur l'existence d'un fichier*

```
lint-docker:
  script:
    - hadolint Dockerfile
  rules:
    - exists:
      - Dockerfile
```

Pourquoi ? Ce job n'a de sens que si le projet contient un Dockerfile. S'il n'existe pas, le job ne sert a rien.

Combiner les regles

Les regles sont evaluees **dans l'ordre** et la premiere qui correspond est appliquee :

```
deploy:
  script:
    - 'echo "Deploiement"'
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
      when: on_success
    - if: $CI_COMMIT_BRANCH == "develop"
      when: manual
    - when: never
```

Lecture : "Si on est sur main, deployer automatiquement. Si on est sur develop, proposer un déploiement manuel. Sinon, ne jamais exécuter ce job."

Etape 3 : Créer le projet sur GitLab

1. Connectez-vous à GitLab
2. Cliquez sur **New project > Create blank project**
3. Nommez le projet `tp6-dynamic-pipelines`
4. Cochez **Initialize repository with a README**
5. Cliquez sur **Create project**
6. Clonez le dépôt en local :

```
git clone https://gitlab.com/VOTRE-UTILISATEUR/tp6-dynamic-pipelines.git
cd tp6-dynamic-pipelines
```

7. Initialisez un projet Node.js :

```
npm init -y
npm install express
npm install --save-dev jest
```

8. Créez un fichier `server.js` minimal :

```
const express = require('express');
const app = express();
const port = process.env.PORT || 3000;

app.get('/', (req, res) => {
  res.json({ message: 'TP6 - Pipelines Dynamiques', status: 'ok' });
});

app.get('/health', (req, res) => {
  res.json({ healthy: true });
});

if (require.main === module) {
  app.listen(port, () => {
    console.log(`Serveur démarre sur le port ${port}`);
  });
}

module.exports = app;
```

9. Créez un fichier `server.test.js` :

```
const app = require('./server');

describe('Server', () => {
  test('app should be defined', () => {
    expect(app).toBeDefined();
  });
});
```

10. Ajoutez le script de test dans `package.json` :

```
{
  "scripts": {
    "test": "jest",
    "start": "node server.js"
  }
}
```

Etape 4 : Créer un pipeline avec des jobs conditionnels (*rules:*)

Creez le fichier `.gitlab-ci.yml` a la racine du projet. Voici les elements cles a comprendre :

Le fichier complet

Copiez le fichier `.gitlab-ci.yml` fourni dans ce TP dans votre projet.

Explications des sections importantes

`workflow:rules` - **Controler quand le pipeline démarre**

```
workflow:
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_TAG
    - when: never
```

Pourquoi ? Sans `workflow:rules`, le pipeline se declenche pour **chaque push** sur **chaque branche**. Cela consomme des ressources inutilement. Ici, on limite le pipeline a trois cas :

- Push sur main (deploiement)
- Creation/mise a jour d'une Merge Request (validation)
- Creation d'un tag (release)
- Tout le reste est ignore (`when: never`)

Variables globales

```
variables:
  NODE_ENV: "test"
  APP_NAME: "tp6-dynamic-app"
  GENERATED_PIPELINE: "generated-pipeline.yml"
```

Pourquoi ? Les variables centralisent la configuration. Si vous changez le nom de l'application, vous le faites a un seul endroit.

Jobs conditionnels

Le job `deploy-production` utilise `rules:` pour ne s'executer que sur main :

```
deploy-production:
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
      when: on_success
    - when: never
```

Le job `deploy-review` utilise `rules:` pour ne s'exécuter que sur les Merge Requests :

```
deploy-review:
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
      when: on_success
    - when: never
```

Pourquoi deux jobs de déploiement ? Parce que le contexte est différent :

- Sur une MR, on déploie un environnement de review temporaire pour tester
- Sur main, on déploie en production

Etape 5 : Créer un pipeline enfant dynamique

C'est la partie la plus puissante de ce TP. L'idée est la suivante :

1. Un job **genere** un fichier YAML pendant l'exécution du pipeline
2. Un autre job **declenche** ce fichier YAML comme un pipeline enfant

Le script de generation

Creez le dossier `scripts/` puis le fichier `scripts/generate-pipeline.js` :

```
mkdir -p scripts
```

Copiez le fichier `scripts/generate-pipeline.js` fourni dans ce TP.

Comment ca marche ?

Le script `Node.js` :

1. Lit les variables d'environnement (`SKIP_INTEGRATION`, `ENABLE_SECURITY_SCAN`, etc.)
2. Construit un objet JavaScript représentant la structure YAML
3. Ecrit le resultat dans un fichier (`generated-pipeline.yml`)

Le job generateur dans `.gitlab-ci.yml`

```
generate-pipeline:
  stage: generate
  script:
    - npm install js-yaml
    - node scripts/generate-pipeline.js
    - 'echo "Pipeline generee avec succes :"'
    - cat generated-pipeline.yml
  artifacts:
    paths:
      - generated-pipeline.yml
```

Pourquoi artifacts ? Le fichier genere doit etre transmis au job suivant. Sans artifact, le fichier serait perdu a la fin du job.

Le job declencheur

```
trigger-dynamic:
  stage: generate
  trigger:
    include:
      - artifact: generated-pipeline.yml
        job: generate-pipeline
  strategy: depend
  needs:
    - generate-pipeline
```

Pourquoi strategy: depend ? Cela fait en sorte que le job parent attende la fin du pipeline enfant et herite de son statut (succes ou echec). Sans cette option, le job parent serait marque comme reussi des le declenchement.

Pourquoi needs: [generate-pipeline] ? Cela cree une dependance explicite : le declenchement ne peut commencer qu'apres la generation. C'est le DAG en action.

Etape 6 : Comprendre `workflow:rules`

`workflow:rules` agit au **niveau du pipeline entier**, pas au niveau d'un job individuel.

Difference entre `rules`: **et** `workflow:rules`

<code>rules</code> : (sur un job)	<code>workflow:rules</code> (global)
Portee	Un seul job
Effet	Inclut ou exclut un job
Position	Dans la definition du job

Exemple concret

```
workflow:
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - when: never
```

Lecture : "Ne creer un pipeline QUE si on est sur main OU si c'est une Merge Request. Dans tous les autres cas, ne pas creer de pipeline du tout."

Pourquoi c'est important ?

Sans workflow:rules, un push sur une branche de feature cree un pipeline inutile. Avec workflow:rules, on economise :

- Du temps de calcul sur les runners
- De la lisibilite dans l'interface GitLab
- Des notifications inutiles aux developpeurs

Etape 7 : Pousser et observer les differents comportements

Test 1 : Push sur une branche de feature

```
git checkout -b feature/test-dynamic
git add .
git commit -m "Ajout du pipeline dynamique TP6"
git push -u origin feature/test-dynamic
```

Resultat attendu : Aucun pipeline ne se declenche (grace a workflow:rules).

Test 2 : Creer une Merge Request

1. Allez sur GitLab
2. Cliquez sur **Create merge request** pour votre branche
3. Validez la creation

Resultat attendu : Un pipeline se declenche avec :

- Les jobs de generation, install, test, build
- Le job deploy-review (pas deploy-production)
- Le pipeline enfant dynamique

Test 3 : Merger sur main

1. Approuvez et fusionnez la Merge Request
2. Observez le pipeline sur main

Resultat attendu : Un pipeline se declenche avec :

- Tous les jobs precedents
- Le job deploy-production (pas deploy-review)

Test 4 : Tester les variables dynamiques

1. Allez dans **Settings > CI/CD > Variables**
2. Ajoutez la variable SKIP_INTEGRATION avec la valeur true
3. Relancez le pipeline

Resultat attendu : Le pipeline enfant dynamique ne contient PAS le job de tests d'integration.

Etape 8 : Utiliser le DAG avec `needs` pour l'optimisation parallele

Qu'est-ce que le DAG ?

Par default, GitLab execute les jobs **stage par stage** : tous les jobs du stage N doivent finir avant que le stage N+1 commence. C'est simple mais pas optimal.

Le **DAG** (Directed Acyclic Graph) permet de definir des dependances **entre jobs individuels**. Un job démarre des que **ses dependances directes** sont terminees, sans attendre la fin du stage entier.

Exemple visuel

Sans DAG (execution par stages) :

```
Stage install: [install] .....
Stage test:    [lint] [unit-test] ...
Stage build:   [build] .....
```

Chaque stage attend le precedent entierement.

Avec DAG (execution optimisee) :

```
[install] --> [lint] -----\
                  \--> [unit-test] --> [build]
```

lint et unit-test démarrent des que install finit, et build démarre des que les deux tests sont finis.

Comment l'utiliser ?

```
lint:
  stage: test
  needs:
    - install
  script:
    - npm run lint

unit-test:
  stage: test
  needs:
    - install
  script:
    - npm test

build:
  stage: build
  needs:
    - lint
    - unit-test
  script:
    - 'echo "Construction de l application"'
```

Pourquoi ? Sur un gros projet, cela peut reduire le temps total du pipeline de 30 a 50% en permettant l'execution parallele.

Etape 9 : Resilience du pipeline

`interruptible: true`

```
unit-test:
  interruptible: true
  script:
    - npm test
```

Pourquoi ? Si vous poussez un nouveau commit pendant que les tests tournent, l'ancien pipeline est **automatiquement annule**. Cela libere le runner pour le nouveau pipeline, plus recent et donc plus pertinent.

`retry: 2`

```
integration-test:
  retry:
    max: 2
    when:
      - runner_system_failure
      - stuck_or_timeout_failure
  script:
    - npm run test:integration
```

Pourquoi ? Certains echecs sont temporaires (probleme reseau, runner surcharge). Plutot que de relancer manuellement, GitLab reessaie automatiquement. On limite les when pour ne pas masquer de vrais bugs.

`timeout: 5m`

```
build:
  timeout: 5m
  script:
    - npm run build
```

Pourquoi ? Un build qui prend normalement 1 minute mais qui tourne depuis 10 minutes est probablement bloqué. Le timeout evite de gaspiller un runner indefiniment.

`resource_group: production`

```
deploy-production:
  resource_group: production
  script:
    - 'echo "Deploiement en production"'
```

Pourquoi ? Si deux pipelines arrivent en meme temps sur main, on ne veut pas deux deploiements simultanes en production. `resource_group` cree un **verrou** : un seul job a la fois peut utiliser la ressource "production". Le second attend que le premier finisse.

Tableau recapitulatif

Fonctionnalite	Mot-cle	Niveau	Utilite
Condition sur un job	<code>rules:if</code>	Job	Executer un job selon une condition
Condition sur les fichiers	<code>rules:changes</code>	Job	Executer un job si certains fichiers ont change
Condition sur l'existence	<code>rules:exists</code>	Job	Executer un job si un fichier existe
Controle du pipeline	<code>workflow:rules</code>	Pipeline	Creer ou bloquer le pipeline entier
Pipeline enfant	<code>trigger:include</code>	Job	Declencher un sous-pipeline genere
DAG	<code>needs:</code>	Job	Definir des dependances directes entre jobs
Annulation automatique	<code>interruptible</code>	Job	Annuler si un nouveau pipeline démarre
Reessai automatique	<code>retry</code>	Job	Reessayer en cas d'echec temporaire
Limite de temps	<code>timeout</code>	Job	Eviter les jobs bloques
Verrou de ressource	<code>resource_group</code>	Job	Empecher l'execution parallele

Erreurs courantes

1. "Pipeline not created" - Aucun pipeline ne se declenche

Cause : `workflow:rules` bloque le pipeline. Verifiez que votre branche ou evenement correspond a une des regles definies.

Solution : Verifiez les conditions dans `workflow:rules`. Pour tester, ajoutez temporairement - `when: always` a la fin.

2. "trigger job failed" - Le pipeline enfant echoue

Cause : Le fichier YAML genere est invalide ou l'artifact n'est pas disponible.

Solution :

- Verifiez que le job `generate-pipeline` a bien l'artifact `generated-pipeline.yml`
- Telechargez l'artifact et verifiez que c'est du YAML valide
- Utilisez le linter CI de GitLab : **CI/CD > Editor > Validate**

3. "needs job not found" - Erreur de DAG

Cause : Le job reference dans `needs:` n'existe pas ou est exclu par ses `rules:`.

Solution : Si un job dans `needs:` peut etre exclu, ajoutez `optional: true` :

```
build:
  needs:
    - job: lint
      optional: true
    - job: unit-test
```

4. Les regles ne se comportent pas comme prévu

Cause : Les regles sont évaluees dans l'ordre et la **premiere qui correspond** est appliquée.

Solution : Mettez les regles les plus spécifiques en premier et terminez toujours par un cas par défaut (- when: never ou - when: on_success).

5. Le script de generation plante

Cause : Le module js-yaml n'est pas installé ou les variables d'environnement ne sont pas définies.

Solution :

- Vérifiez que `npm install js-yaml` est bien dans le script du job
- Les variables non définies valent `undefined` en JavaScript, gérez ce cas

6. "resource_group" bloque le pipeline

Cause : Un précédent déploiement est encore en cours ou bloqué.

Solution : Allez dans **CI/CD > Pipelines**, trouvez le job bloqué et annulez-le manuellement.

Pour aller plus loin

- Documentation officielle : [Dynamic child pipelines](https://docs.gitlab.com/ee/ci/pipelines/downstream_pipelines.html)
- Documentation officielle : [rules](<https://docs.gitlab.com/ee/ci/yaml/#rules>)
- Documentation officielle : [DAG](https://docs.gitlab.com/ee/ci/directed_acyclic_graph/)
- Essayez de créer un pipeline qui génère des jobs différents selon les dossiers modifiés (monorepo)